applied sciences

IMPACT FACTOR 2.5

CITESCORE 5.3

Article

Federated Learning in Dynamic and Heterogeneous Environments: Advantages, Performances, and Privacy Problems

Fabio Liberti, Davide Berardi and Barbara Martini

Special Issue

Advancements in Multi-Agent Systems and Artificial Intelligence: Methodologies, Applications, and Future Trends

Edited by
Dr. André Sales Mendes

MDPI

https://doi.org/10.3390/app14188490

applied sciences

MDPI

Article

Federated Learning in Dynamic and Heterogeneous Environments: Advantages, Performances, and Privacy Problems

Fabio Liberti, Davide Berardi * and Barbara Martini

Department of Science and Engineering, Universitas Mercatorum, 00186 Rome, Italy; fabio.liberti@studenti.unimercatorum.it (F.L.); barbara.martini@unimercatorum.it (B.M.)
* Correspondence: davide.berardi@unimercatorum.it

Abstract: Federated Learning (FL) represents a promising distributed learning methodology particularly suitable for dynamic and heterogeneous environments characterized by the presence of Internet of Things (IoT) devices and Edge Computing infrastructures. In this context, FL allows you to train machine learning models directly on edge devices, mitigating data privacy concerns and reducing latency due to transmitting data to central servers. However, the heterogeneity of computational resources, the variability of network connections, and the mobility of IoT devices pose significant challenges to the efficient implementation of FL. This work explores advanced techniques for dynamic model adaptation and heterogeneous data management in edge computing scenarios, proposing innovative solutions to improve the robustness and efficiency of federated learning. We present an innovative solution based on Kubernetes which enables the fast application of FL models to Heterogeneous Architectures. Experimental results demonstrate that our proposals can improve the performance of FL in IoT and edge environments, offering new perspectives for the practical implementation of decentralized intelligent systems.

Keywords: privacy; federated machine learning; edge computing; ubiquitous intelligence

check for updates

Citation: Liberti, F.; Berardi, D.; Martini, B. Federated Learning in Dynamic and Heterogeneous Environments: Advantages, Performances, and Privacy Problems. *Appl. Sci.* **2024**, *14*, 8490. https://doi.org/10.3390/app14188490

Academic Editor: Antonio Pagliaro

Received: 27 August 2024
Revised: 15 September 2024
Accepted: 18 September 2024
Published: 20 September 2024

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Federated Learning (FL) is a new approach to machine learning that allows you to train models on data distributed across different locations without the need to centralize them. In this way, the information contained in large quantities of data can be exploited without compromising the privacy and security of the data themselves. Federated Learning (FL) proves to be an ideal paradigm to collaboratively exploit the enormous amounts of data from devices located around the world, the Internet of Things (IoT), while preserving privacy and security. Federated learning (FL) and edge computing are complementary technologies for distributed AI. FL allows edge devices to train models collaboratively, without sending all the data to the cloud. This reduces latency, improves privacy, and increases computational efficiency. In this work we focus on addressing the following Research Questions through the implementation of a distributed and federated machine learning software suite (track changes and periodical updates are available on the repository: https://github.com/FabioLiberti/DHFLPL, accessed on 14 September 2024):

Q1: *What are the impacts of heterogeneous environments in Federated Learning?*

Q2: *How can the federated learning scenario benefit from different systems and architectures?*

Q3: *What are the privacy implications of using federated learning?*

Q4: *How can a cluster management system (i.e., Kubernetes) make the application of heterogeneous machine learning models feasible and easier to conduct?*

We argue that, by replying to these research questions, we could create a clear view of the distributed machine learning scenario. This would highlight the weak points and strengths of such methods.

We exploited various methods to generate this kind of system. While distributed and heterogeneous federated learning covers multiple fields such as dedicated algorithms, different architectures and model management, we focus on the analysis of using a federated learning platform with heterogeneous computer architectures. Leveraging cluster management tools, we focused on the analysis of simulating a cluster of devices, which participate in the training of a distributed dataset. This participant uses common methods and models, with the possibility of guaranteeing privacy using a differential privacy algorithm. We explored a number of solutions, which are presented here. To the best of our knowledge, exploring related and similar works is a novel approach to the theme of federated learning. Simulating heterogeneous nodes using Kubernetes clusters and discussing performance outcomes of the combination of ARM and Intel devices.

Federated Learning and Edge Computing in the Internet of Things

Ubiquitous Intelligence. Ubiquitous intelligence refers to intelligent systems that are pervasive and ubiquitous, and integrated into the surrounding environment. These systems use sensors, mobile devices, advanced network technologies and artificial intelligence algorithms to provide advanced services and functionality without requiring direct user intervention. The goal is to create an intelligent environment that can anticipate user needs and respond proactively, improving the overall experience. This pervasive intelligence is fundamental in contexts such as smart cities, the Internet of Things (IoT) and new generation networks such as 6G, where the ability to provide services in real-time and in a distributed manner is essential [1,2].

Containers in Federated Learning. Using containers, such as Docker and Kubernetes, in Federated Learning offers numerous benefits. Docker allows you to create isolated, reproducible environments for each participating node, ensuring consistency and ease of deployment. Kubernetes, on the other hand, facilitates the management of containers at scale, automating the deployment, scaling, and operation of containerized applications. This approach increases the scalability and resilience of the Federated Learning system, allowing it to easily manage many distributed nodes. Additionally, containers improve security by isolating processes and limiting access to resources. Integration with Continuous Integration/Continuous Deployment (CI/CD) tools further simplifies system upgrades and maintenance, ensuring that all participants are running the latest software versions [1,3]. These advantages make containers an ideal solution for implementing Federated Learning in heterogeneous, large-scale environments.

2. Related Work

2.1. Dynamic Federated Learning

Federated learning (FL) is particularly useful in dynamic environments where data and models evolve over time. However, this scenario presents unique challenges and opportunities. For instance, Models must adapt quickly to new data and changes in user behavior, we need to track down these changes and noises. Also, data quality and device availability are quite difficult to foresee; therefore, the system must address these details. As described by Research Questions in Section 1, privacy must be preserved even in a fast-moving environment.

Aside from that, federated learning comes with different opportunities and is the key enabler technology for the following: Models can be continuously updated with new data, improving their performance over time and in real time. Also, distributing the computing load improves the overall efficiency [3]. To wrap up: FL in dynamic environments is an evolving research area with the potential to transform AI in industries such as healthcare, communications, finance, and the Internet of Things (IoT).

Projects such as [4] are similar to our proposal but do not tackle the complexity of having heterogeneous environments such as different architectures or models. For instance, Kim et al. in [5] employ a system that is built using high-performance commercial hardware such as Nvidia Jetson. Technologies enabled by this approach are also analyzed

by Pham et al. in [6]; however, the work is focused more on the enabled scaling offered by Kubernetes than on the advantages or heterogeneity of the devices or models. The allocation of resources is covered in [7], in this work by Nikolaidis et al., an in-depth analysis of the allocation of resources is presented; we focus on a slightly relaxed problem, avoiding the allocation of tasks but focusing on the performance of heterogeneous device development without complex scheduling between them. The combination of these analyses could be valid in future work. Another interesting project is presented in [8] by Jayaram et al., which focuses on the employment of an aggregator that runs on Kubernetes, focusing on resource distribution and aggregation. We explored the possibility of having a simpler distribution method without evaluating the complex algorithms that can move the results of our analysis from the heterogeneity of the devices.

Privacy is an explored field in the federated learning analysis, for instance, projects such as [9] include modern technologies such as homomorphic encryption in the field. While being complex in terms of implementation and management, we focus on simpler behaviors, based on the data themselves. Homomorphic encryption is also a computation-intensive task, making it difficult to implement on mobile or IoT devices. Differential privacy is explored in projects such as [10,11] and collected in surveys such as [12], which covers techniques used in 2022. From these works and their promising results, we took inspiration to introduce differential privacy into federated learning and analyze it in heterogeneous architecture environments.

Federated learning also covers different infrastructures with different setups such as serverless infrastructures [13]. While these infrastructures are focused on the maximization of resource usage, it is not easy to port them to different architectures or to configure them to perform in heterogeneous environments.

On the other hand, technology enablers are analyzed in depth in the field, for instance, works such as [14] cover the analysis of Kubernetes applied to blockchain-based federated learning. We do not want to tie our system to a specific technology such as Blockchain or DHT, making it simpler and focusing the effort of our work on an analysis of the behavior of heterogeneous architectures in Federated Learning scenarios. This is also the case of vertical applications such as 5G networks and beyond such as 6G networks [15,16]. These are very specific scenarios that cover the details of the infrastructure management or the learning procedure, we focus on broad term learning, covering image classification in the test but not limiting our system to it.

2.2. Heterogeneous Federated Learning

Using Federated learning we need to address heterogeneity in several dimensions: communication, models, statistics, and devices.

Communication: It can be difficult to coexist with Networks with variable bandwidth, latency, and reliability. These are difficult to manage and correct. For instance, protocols such as TCP/IP are compatible with changing the reliability of the communication, but not so elastic in terms of infrastructural changes such as IP or handover [17].

Models: Different models, machine-learning architectures, data formats, and data dimensions can create difficulties in aggregation. For instance, two distinct layouts of neural networks can generate different results with the same data. In this case, the results need to be evaluated and analyzed with a specific algorithm [18].

Statistics: Non-uniform data distributions and misaligned statistics can affect the quality of the overall model. For instance, dealing with heterogeneous data from different sources can be difficult and needs to be adapted with specific algorithms and preparations, which otherwise can lead to over-fit and under-fit problems in the models.

Devices: Limitations in computing power, memory, and communication capabilities between devices can slow the learning process [2]. For instance, distributing the analysis between low-powered end devices such as smartphones or low-end IoT devices and high-end processing servers could slow down the high-end devices.

Despite these challenges, FL offers opportunities to exploit data diversity and improve model robustness. Techniques such as data compression, differential federated learning, and model adaptation can mitigate heterogeneity issues. As described by Section 1 FL in heterogeneous environments is an active research field with the potential to advance distributed and adaptive artificial intelligence in complex contexts. We argue that, to the best of our knowledge, the integration of heterogeneous environments and cloud-native infrastructures such as Kubernetes or Docker is a novel approach.

2.3. Privacy Leakage Problem

One of the crucial challenges in Federated Learning, as described by Section 1, particularly in heterogeneous environments, is ensuring data privacy. While FL is designed to keep data decentralized and local to each participating customer, the variability and complexity of these environments introduce several channels which sensitive information can be inadvertently exposed through. This phenomenon, known as Privacy Leak, can occur through gradient reversal attacks [19], exploited model updates [20], side-channel attacks [21], and membership inference attacks [22]. Understanding and mitigating these risks is critical to the safe and effective implementation of FL, especially in sensitive industries where the diversity of devices and data sources and their marked instability and variation in the frequency of device participation in the federated learning process further complicate the privacy landscape. For this we propose the following Use Case Scenario: let us suppose to have different users, which connect to a platform through smartphones. These smartphones can obtain details on the environment such as photos of the environment and details on the surroundings such as road traffic. This can be sent to a centralized server to analyze it and suggest, for instance, the best road to take to avoid traffic.

3. Method

Problem Statement and Motivation

Federated Learning (FL) is designed to enable decentralized model training across multiple devices, ideally while preserving data privacy. However, deploying FL in dynamic and heterogeneous environments introduces several unique challenges. These environments are characterized by diverse device capabilities, varying network conditions, and non-IID (independently and identically distributed) data across devices [23].

4. Statistical Methodology for Federated Learning

The use of distributed computation, enabled by techniques described in this paper, allows for efficient resource management and optimal scalability in dynamic and heterogeneous environments. Descriptive statistics such as the mean μ and variance σ^2 are calculated locally on each node:

$$\mu_i = \frac{1}{n_i}\sum_{j=1}^{n_i} x_j, \quad \sigma_i^2 = \frac{1}{n_i - 1}\sum_{j=1}^{n_i}(x_j - \mu_i)^2 \tag{1}$$

where x_{ij} represents the local data on node i and n_i is the number of samples. Distributed and federated computations ensure these calculations are performed in isolated and consistent environments, allowing for the secure and accurate aggregation of local models. The global mean μ_g and global variance σ_g^2 can then be calculated as:

$$\mu_g = \frac{\sum_{i=1}^{k} n_i \mu_i}{\sum_{i=1}^{k} n_i}, \quad \sigma_g^2 = \frac{\sum_{i=1}^{k}(n_i - 1)\sigma_i^2}{\sum_{i=1}^{k} n_i - k} \tag{2}$$

where k is the total number of participating nodes. This approach ensures robustness and flexibility in Federated Learning.

4.1. Federated Averaging

The Federated Averaging (FedAvg) algorithm is a key methodology in the field of federated learning [24], a distributed learning paradigm that allows the training of machine learning models without having to centralize the data. In this approach, several devices or clients, which can be smartphones, IoT sensors, or other edge computing entities, locally train a copy of the model using their own private data. The FedAvg process works using five macro-points:

1. Initialization: The central server initializes the global model with initial weights and distributes it to all participating clients.
2. Local Training: Each client receives the global model and trains it locally on its data for a predefined number of epochs or iterations. This local training produces an updated set of weights specific to each client.
3. Sending Weights: Clients send their updated weights to the central server. During this process, only the model weights are transferred, while the raw data remain on local devices, thus preserving user privacy in a broad way.
4. Aggregation: The central server collects all updated weights from clients and calculates the weighted average of these weights to update the global model. Weighting can take into account the amount of local training data from each client, ensuring that clients with more data have a greater influence on updating the model.
5. Iteration: This process of deploying the global model, training locally, dispatching weights, and aggregating is repeated for a predefined number of rounds until the global model reaches a desired convergence or performance level.

This is described with more detail—such as averaging formula—in Algorithm 1. The Federated Averaging algorithm's formula is

$$w_{t+1} = \sum_{k=1}^{K} \frac{n_k}{n} w_{t+1}^k$$

where w_{t+1} is the weight vector of the updated global model; K is the total number of participating devices; n_k is the number of data samples on the device k; $n = \sum_{k=1}^{K} n_k$ is the total number of data samples across all devices; w_{t+1}^k is the weight vector of the local model updated by the device k per round $t+1$.

This approach is particularly effective in heterogeneous and dynamic environments, such as those characterized by the use of IoT devices and edge computing described in this paper, as it reduces the need to transfer large volumes of data across the network and addresses issues related to data privacy and security data. Furthermore, the aggregation of weights allows to obtain a robust and generalized global model, exploiting the diversity of the data distributed among clients. The privacy is not automatically maintained by default, to introduce it, we need to implement advanced techniques such as Differential Privacy, which we will highlight in detail in Section 4.2.

Algorithm 1: FedAvg: Federated Averaging Algorithm

Data: Initial global model w_0, number of clients K, number of rounds T
Result: Updated global model w_T
for *each round* $t = 1, 2, \ldots, T$ **do**
  for *each client* $k = 1, 2, \ldots, K$ ***in parallel*** **do**
    $w_{t+1}^k \leftarrow$ Client Update(k, w_t) ; `// Client k update the model`
  $w_{t+1} \leftarrow \frac{1}{K} \sum_{k=1}^{K} w_{t+1}^k$; `// Weight Averaging`

4.2. Privacy Leakage

Attackers can exploit the privacy of the system with different tactics. Each of these tactics can lead to different results and details. This is particularly important in the field of distributed federated learning, for instance, if the recent development of Recall (https://learn.microsoft.

com/en-us/windows/ai/apis/recall, accessed on 14 September 2024), an artificial intelligence model of Microsoft, which can be used to search and retrieve images using details of their content, expressed in natural language. If systems similar to this one use federated machine learning and send data to a central analyzer, privacy must absolutely be one of the core features of the system to avoid leakage of personal data of users.

Gradient Inversion Attacks: Attackers can reconstruct original data from the shared gradients during the model update process. Even though raw data are not directly shared, gradients can carry enough information to reveal sensitive data points. For example, given a gradient $\nabla L(\mathbf{w}, \mathbf{x})$ computed with respect to the model weights $\mathbf{w}$ and input data $\mathbf{x}$, it is possible to approximate $\mathbf{x}$ by minimizing the difference between the computed gradient and the gradient of a guessed input.

Model Updates: Repeatedly sharing model updates can lead to leakage of information about the training data. Over time, these updates can accumulate enough information for an attacker to infer private data. For instance, in a typical FedAvg setup, the global model update at round $t+1$ is computed as:

$$w_{t+1} = w_t + \eta \sum_{k=1}^{K} \frac{n_k}{n} \Delta w_t^k \tag{3}$$

where η is the learning rate, n_k is the number of data points at client k, n is the total number of data points, and $\Delta \mathbf{w}_t^k$ is the local model update from client k. If the model update is performed within a short period of time, the attacker can infer from the absence or the high volume of sent data the position or the details on the data of the user. Otherwise, if the model is not sufficiently protected in transit, the attacker can intercept it and try to analyze it.

Side-Channel Attacks: Attackers can exploit side-channel information, such as the timing or size of the communications between clients and the server, to gain insights into the data being processed. These side channels can indirectly leak sensitive information even if the data and model updates are encrypted. Famous attacks such as Spectre and Meltdown fall in this category.

Membership Inference Attacks: These attacks aim to determine whether a specific data point was part of the training dataset. This is particularly concerning in scenarios where the presence of certain data points can imply sensitive information. Given a model $\mathcal{M}$ and a data point $\mathbf{x}$, an attacker can train a shadow model to infer the membership status of $\mathbf{x}$ by analyzing the output confidence scores.

To mitigate these risks, several techniques have been proposed. The main and most present in literature are as follows:

Differential Privacy: A technique that adds noise to the gradients or the model parameters to mask the contribution of individual data points [25]. By ensuring that the inclusion or exclusion of a single data point does not significantly affect the output, differential privacy provides a strong privacy guarantee. The formal definition of differential privacy is as follows:

$$\Pr[\mathcal{M}(D) \in S] \leq e^{\epsilon} \Pr[\mathcal{M}(D') \in S] + \delta \tag{4}$$

where $\mathcal{M}$ is the randomized mechanism, D and D' are datasets differing by one element, S is a subset of possible outputs, ϵ is the privacy budget, and δ is a small probability.

Secure Multi-Party Computation (SMPC): SMPC protocols [26] allow multiple parties to jointly compute a function over their inputs while keeping those inputs private. In the context of FL, SMPC can be used to securely aggregate model updates without exposing individual contributions. For example, using additive secret sharing, each

client k splits its update $\Delta \mathbf{w}_t^k$ into shares and distributes them among the clients. The server only receives the aggregated result, which is the sum of all shares.

Homomorphic Encryption: This allows computations to be performed on encrypted data without needing to decrypt them first. In FL, homomorphic encryption can be used to perform model aggregation securely, ensuring that the server does not see the raw updates from clients. Given an encryption function E and a decryption function D, homomorphic encryption ensures that:

$$D(E(a) \cdot E(b)) = a + b \tag{5}$$

Unfortunately, at the moment of writing, Homomorphic Encryption is orders of magnitude slower than non-homomorphic alternatives, making it unfeasible for large amounts of data.

5. Application of Multilayer Architecture

To address the challenges exposed in the previous sections, we propose to use a virtual machine (VM) cloud environment with Docker and Kubernetes (specifically its implementation called k3s) containers. This approach offers a number of advantages, which map to the following:

Scalability: The cloud environment can be easily scaled to meet changing needs for computational resources. By leveraging cloud platforms, resources can be dynamically allocated based on current demand, allowing for seamless expansion or contraction of the infrastructure. This is particularly useful in federated learning (FL), where the number of participating devices and the volume of data can vary significantly over time. Autoscaling features in Kubernetes ensure that container instances are automatically adjusted to handle fluctuating workloads, maintaining optimal performance without manual intervention.

Flexibility: Containers can be used to isolate and manage different components of the FL system, making it easy to adapt to heterogeneous devices and data. Each container can be configured with the specific dependencies and environment required for a particular task, ensuring consistency and reproducibility across different nodes. This modular approach allows for the rapid deployment of updates and new features, as well as easier troubleshooting and maintenance. Furthermore, containers enable the use of diverse programming languages and tools within the same FL system, enhancing the ability to integrate with various data sources and device capabilities.

Reliability: The cloud environment provides a reliable and resilient infrastructure that can tolerate node failures and other issues. Cloud providers typically offer robust service level agreements (SLAs) and fault-tolerant architectures that ensure high availability. Docker and Kubernetes contribute to this reliability by managing container health checks, restarts, and failovers. In the event of a node failure, Kubernetes can seamlessly migrate workloads to other healthy nodes, minimizing downtime and maintaining the continuity of the FL process. Additionally, the use of multi-zone or multi-region deployments can further enhance the resilience of the system against localized failures [27].

In detail, the proposed architecture is composed of different building blocks. The main blocks are pictured in Figure 1 and are the following:

A cluster of VMs in the cloud: VMs provide the isolation and computational resources needed to run containers. Each VM can host multiple containers, offering a layer of abstraction that separates the hardware from the application layer. This isolation ensures that each container operates in a controlled environment, preventing conflicts and enhancing security. By distributing containers across multiple VMs, the system can leverage the cloud's elasticity to optimize resource utilization and cost efficiency.

A container orchestrator (Docker and Kubernetes): The orchestrator manages the lifecycle of containers, ensuring they are always running and automatically scaling them as needed. Docker provides the containerization platform, while Kubernetes (k3s is a lightweight Kubernetes distribution designed for resource-constrained environments such as edge computing and IoT devices, it simplifies Kubernetes by reducing dependencies and the overall binary size) handles the orchestration. These tools automate the deployment, scaling, and operation of application containers across clusters of hosts. Kubernetes' advanced scheduling capabilities ensure that containers are optimally placed based on resource requirements and constraints, improving efficiency and performance.

A federated learning module: The FL module handles communication between participants, model aggregation, and local model updating. This module is responsible for coordinating the training process, ensuring that updates from local models are securely and accurately aggregated to form a global model. It manages the distribution of the global model to clients, collects local updates, and performs federated averaging or other aggregation techniques. The module also handles encryption and secure communication protocols to protect the integrity and confidentiality of the data being transmitted.

A data management module: The data management module pre-processes the data, distributes them to participants, and ensures data privacy. This module is crucial for handling the diverse and often sensitive nature of the data used in FL. It includes functionalities for data normalization, anonymization, and encryption to comply with privacy regulations and protect user information. The module also manages the allocation of data to ensure balanced and representative training across all participants, improving the robustness and fairness of the final model.

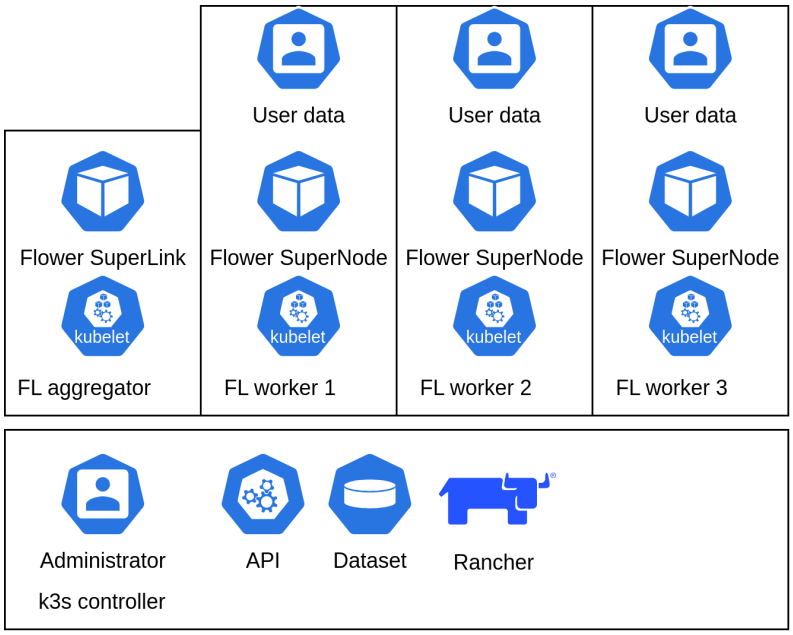

Figure 1. Schema of implementation using k3s. The k3s worker can be any architecture compatible with Kubernetes, with heterogeneous architecture. For instance, in our implementation, k3s workers are ARM virtual machines running in a cloud provider (Oracle).

This approach offers a flexible, scalable, and reliable solution to address the challenges of Federated Learning in dynamic and heterogeneous environments. The use of a cloud and container environment simplifies the management of the system and adapts it to different needs. With this approach, you can leverage the benefits of Federated Learning to develop effective, privacy-preserving machine learning models in complex, ever-changing

environments. The integration of advanced container orchestration and cloud capabilities not only enhances the scalability and flexibility of the system but also ensures robustness and reliability, making it well-suited for a wide range of applications where data privacy and security are priorities.

5.1. Architectural Support Using Docker and Kubernetes

With the application of this multilayer architecture in dynamic and heterogeneous environments, the use of container technologies like Docker and Kubernetes (k3s) plays a crucial role in mitigating privacy leakage:

- Isolation and Consistency: Containers provide isolated environments for each participating node, ensuring that computations are consistent and reproducible. This isolation limits the potential for privacy leakage between nodes.
- Orchestration and Scaling: Kubernetes automates the deployment, scaling, and operation of containers. This ensures efficient resource management and helps in dynamically adapting to changing computational demands, which is essential in large-scale FL systems.
- Secure Communication: Containers can be configured to enforce secure communication protocols, ensuring that data in transit are protected. Kubernetes can manage and automate the deployment of these secure channels.
- Automated Updates: Integration with Continuous Integration/Continuous Deployment (CI/CD) tools ensure that security patches and updates are consistently applied across all nodes, reducing vulnerabilities.

5.2. Implementation

The system was implemented and analyzed using Oracle Cloud Infrastructure (OCI) (https://www.oracle.com/it/cloud/, accessed on 14 September 2024). This infrastructure offers x86-based virtual machines and ARM64-based virtual machines. We initially implemented our cluster using three ARM-based virtual machines and connected them using a private network. That is, after that we installed the k3s controller and rancher in one of the machines, using the remaining two as k3s workers. To facilitate the usage of the cluster, we also instantiated rancher inside the cluster. Rancher is a Kubernetes management platform, which enables the administrator to graphically analyze and use the cluster without dealing with advanced Kubernetes configurations from the command line. This enabled us to automatically manage the infrastructure, adding and removing nodes easily. To achieve the federated learning and to run the models we installed Flower [28] inside the container images. This tool, which is a framework for creating federated learning models, removes the daunting task of adapting our model to the updates sent by the users. This is implemented by using two docker containers: flower-superlink, the main module that aggregates the workers and flower-supernode, the worker itself that runs the data-gathering and model update application. The implementation is pictured in Figure 1. In this picture, the containers, the infrastructure and the controller are visible. The controller possesses its initial dataset which can be distributed to the users (in production the dataset will be augmented with user personal data or private dataset). The nodes use the flower as the machine learning platform to analyze the data and aggregate the result to the flower superlink. Obviously, more pods than one per worker can be instantiated. The user can then access the superlink to obtain the result of the learning and the outcome of the learning process.

The dataset is non-IID distributed over the machines using Flower. That is, every node obtains a subset of data that is distributed in a non-identical fashion. In Listing 1 the code extract that performs the distribution is present and the data are equally distributed over the nodes.

Listing 1. Dataset Distribution Code.

```
def split_data_for_federated_learning(data, num_clients):
    data_per_client = len(data[0]) // num_clients
    datasets = [(data[0][i*data_per_client:(i+1)*data_per_client],
                 data[1][i*data_per_client:(i+1)*data_per_client])
                for i in range(num_clients)]
    return datasets
```

The data are computed on the various nodes and sent to a super link using Flower. This approach enables the distribution to an arbitrary number of nodes, without associating directly the data with the nodes. Before this, the data are randomly shuffled to avoid overfitting due to sorted data.

After this step, the training is performed, every client then independently analyzes part of the dataset, training its internal models with the provided data. That is, the client analyzes the data and sends out the weight to the main collector using flower Superlink. In Listing 2 we present this extract of code, conducting exactly this part of implementation:

Listing 2. Federated Learning Algorithm.

```
for i, (x_train, y_train) in enumerate(federated_datasets):
    client_models[i].fit(x_train, y_train, epochs=1, verbose=0)
    client_weights.append(client_models[i].get_weights())
```

Line 1 runs over all the provided datasets for every client, analyzes them and retrieves the updated weight. The epoch is set to 1 due to the fact that these runs are replicated for the number of desired epochs and put in a for loop.

The updated weights are then sent to the various federated clients, updating every one of them, and providing an updated model to every one of them. Using this technique, the nodes can have an updated and performant model without requiring specialized hardware or an enormous amount of resources such as GPUs, CPUs, RAM, secondary memory support, money, or time.

Finally, the updated weights are centralized and checked over a part of the dataset called "test data". This part is to check the validity of the method and the performance of the federated learning procedure. These data are processed as a reference using the code presented in Listing 3.

Listing 3. Validation of the provided model analyzing performance indicators such as precision and recall.

```
avg_model = create_model(input_shape, num_classes)
avg_model.set_weights(new_weights)
loss, acc = avg_model.evaluate(test_data[0], test_data[1])
precision, recall, f1 = \
  evaluate_with_metrics(avg_model, test_data[0], test_data[1])
```

This code recreates a model as another node of the federated learning platform. Then, it proceeds to update the weights with the one retrieved from the distributed computation. After this step, the module is evaluated as a single-node computation.

To ensure the flexibility of the system, we automatized the registration to the flower platform. That is, when a Kubernetes container is powered up in a pod, it will participate automatically in the next round of distributed federated learning evaluation independently from its architecture. This was strongly automatized by the Kubernetes scale and replica features, enabling the possibility to increase the number of virtualized nodes with a single command kubectl scale –replicas=50 fl/client. Also, it is possible to perform automatic scaling, based on the performance of the system. For instance, we created a new metric using precision as the reference value. When the precision is too low (less than 50%),

we create a new node, automatically trying to increase the precision due to having more workers. This could lead to cost-effective machine learning, where performances dictate the strict requirements. Setting a threshold to the minimum precision, the system will auto-adapt to reach the required value for the metric.

To maintain privacy and avoid privacy leaks, we implemented Differential Privacy algorithms. Using pydp, a Python library implementing differential privacy, we performed two actions: (1) First we redacted the private data of the users, such as photos tagged as personal and data such as email addresses, phone numbers, home addresses and so on; if an application wants to avoid privacy leaks, the blocklist provided with the code should be implemented; (2) then, the dataset is augmented with noise, to avoid the recognition of private values in terms of numbers, for instance that can be used in Electronic Medical Records or Account Statement.

Results

We tested our implementation with varying numbers of clients from two to fifty and over five different datasets, which are well-known and open to the public. The datasets we considered are as follows:

CIFAR-10: An image dataset with 10 classes, available at https://www.cs.toronto.edu/~kriz/cifar.html (accessed on 14 September 2024).

We selected this dataset as the main reference, due to the fact the development has been made with it in mind. We expected this dataset to be the best behaving among other well-known datasets.

CIFAR-100: Same as the CIFAR-10 dataset but with 100 classes. This is harder to train due to the augmented details of the classes, due to this fact, we expected the system to have lower performance.

MNIST: A dataset of hand-written digits, therefore, composing 10 classes, similar to CIFAR-10. Available at http://yann.lecun.com/exdb/mnist/ (accessed on 14 September 2024) we expect it to have performances similar to the CIFAR-10 ones.

Fashion-MNIST: A dataset interchangeable with MNIST, taken as a reference, it is composed of 10 classes of photos of dresses. We expect it to have comparable performance to MNIST.

SVHN: A dataset composed of House Street Numbers, obtained with Google Street View. Similar to MNIST, it is composed of 10 classes of digits. Available at http://ufldl.stanford.edu/housenumbers/ (accessed on 14 September 2024).

We selected comparable datasets. In terms of the number of items used for the training set and the number of elements used for validation. Also, the number of classes is comparable to check if the system is over or under-fitting in particular datasets. Finally, we analyzed CIFAR-100, to check how the system behaves when used with a dataset with significantly more classes.

The tests are run for 150 epochs, which requires a variable time and obtaining variable results, which range from 99.996% Accuracy (MNIST dataset with two clients) to 35.53% accuracy in the case of CIFAR-100 and 50 clients.

In Figure 2 we present the accuracy and loss of our system over these variations. It is well known that accuracy in this field is still not comparable with the centralized one at the moment of writing as clearly seen in the plots. Comparing our results with the ones present in literature [29] we clearly see a similar progression even in heterogeneous architecture, without significant lacking in terms of performances. This comparison is pictured in Figure 3. From the data of main results of Figure 2, it is analyzable that the results vary a lot in terms of datasets—especially when the number of classes rises—but are not influenced by the number of federated clients. The results of not simple datasets (i.e., not MNIST)—especially when increasing the number of classes—are not comparable with the results obtained with the centralized approach, present in the literature and almost close to 99% accuracy.

This is due to the nature of federated learning, in which local minimum values are not so easy to catch by the learning process. Focusing on modern optimization such as the one described by Wang et al. in [30] it can be possible to increase the performance of the network, this improvement is still under development.

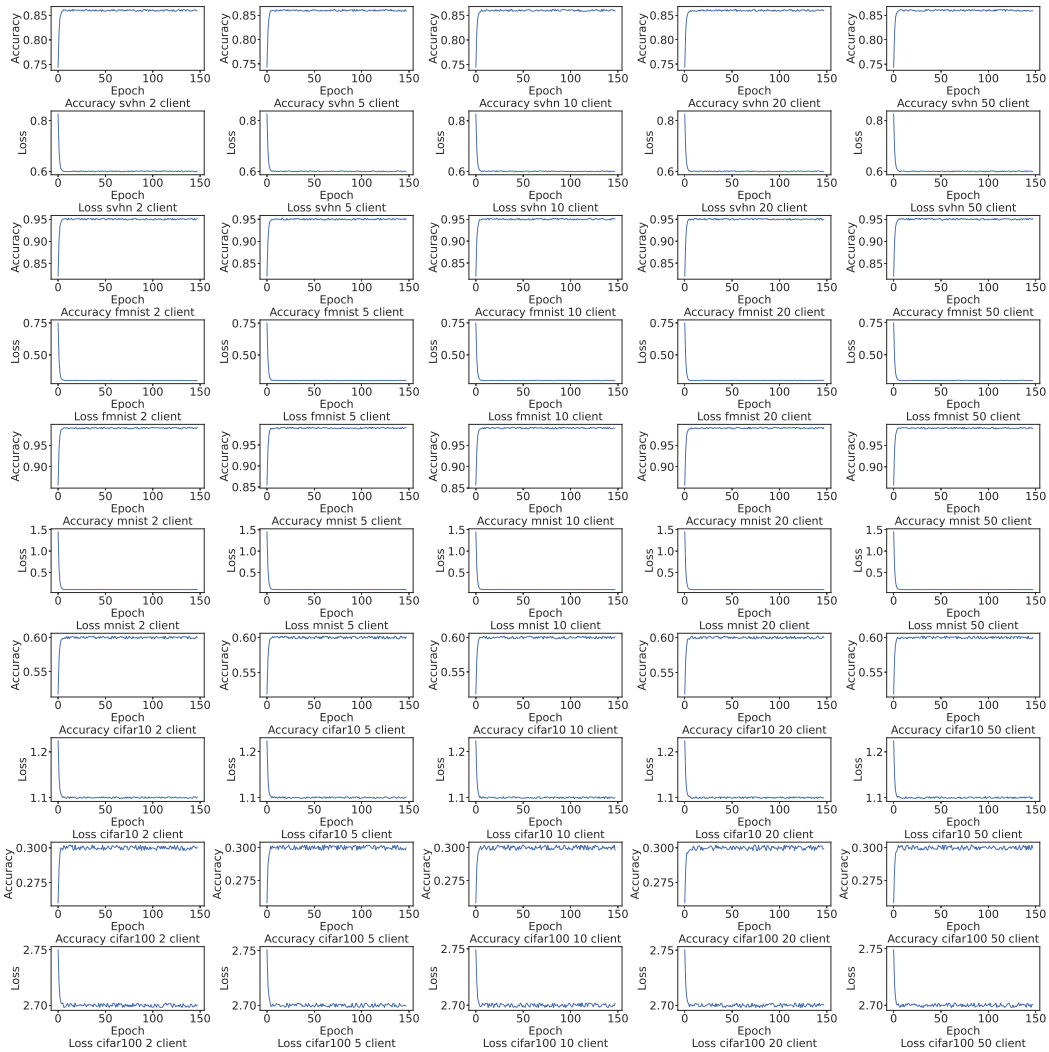

Figure 2. Analysis of the performance of the system over 150 epochs in terms of accuracy and loss over five different datasets and with clients varying from two to fifty.

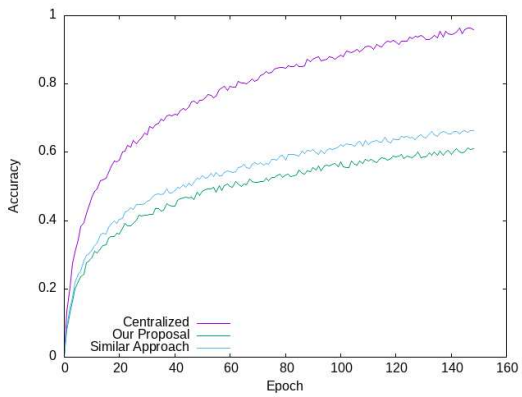

Figure 3. Comparison of the accuracy of federated and centralized learning systems with similar works.

6. Final Considerations and Future Directions

Advantages. The use of containers in a federated learning environment provides numerous benefits. Containers facilitate the easy deployment and management of applications by encapsulating all the necessary dependencies and configurations. This ensures consistency across different environments, reducing the chances of conflicts and errors. Furthermore, containers support rapid scaling and resource allocation, which is critical in handling the dynamic workloads typical of federated learning systems. The isolation provided by containers also enhances security by limiting the potential impact of vulnerabilities within any single container.

Disadvantages. Despite their benefits, containers also present some challenges. One of the main disadvantages is the complexity involved in managing a large number of containers, especially in distributed environments. This requires sophisticated orchestration tools like Kubernetes, which can have a steep learning curve. Additionally, while containers improve resource utilization, they still introduce overhead compared to running applications directly on the host OS, potentially affecting performance. The security of containers is an ongoing concern, as misconfigurations or vulnerabilities can lead to breaches if not properly managed.

Unresolved Issues. Several unresolved issues remain in the deployment of federated learning using containers. One significant challenge is ensuring data privacy and security across distributed nodes, particularly when dealing with sensitive information. While encryption and secure communication protocols help, they do not eliminate all risks. Another issue is the heterogeneity of participating devices, which can vary widely in terms of computational power, connectivity, and data quality. Balancing the load and ensuring fair contribution from all nodes is an ongoing area of research. Additionally, managing the orchestration of containers in highly dynamic environments with frequent changes in node availability remains complex. Also, increasing the performance of the presented model can be challenging due to the distribution of data, leading to undesirably low performances, compared to the centralized architecture.

Further Developments. Looking ahead, several developments could enhance the deployment and effectiveness of federated learning in containerized environments. One promising area is the use of edge computing, where computation is performed closer to the data source, reducing latency and bandwidth usage. Integrating advanced AI and machine learning techniques to optimize resource allocation and orchestration decisions could also improve efficiency and performance. Additionally, developing more robust security frameworks specifically designed for federated learning can address some of the current privacy and security challenges. The evolution of federated learning algorithms that can better handle heterogeneous and dynamic environments, such as those incorporating reinforcement learning for adaptive learning rates and model updates, is another crucial area. Finally, enhancing interoperability standards among different container orchestration platforms can simplify the integration of various systems and improve overall flexibility.

Another important aspect of the use of cluster management platforms to tackle this problem is automatic scaling. In this paper, we presented an automated scaling platform based on Kubernetes, which we also attached to a custom metric. This enabled us to automatically scale the number of nodes—containers in this case—based on the required precision. This could lead to a cost-effective platform for federated learning, which is currently being studied and developed by the team.

Conclusions

In this work, we presented a distributed and heterogeneous infrastructure to test, manage, and create federated learning scenarios. This approach is promising but still lacks performance and stable approaches to deal with heterogeneous systems. In conclusion, we reply to the Research Questions presented in Section 1. During the entire paper, we presented scenarios, implementation details and downsides of this approach, replying to Q1 and Q2. Federated learning is a promising technology and can be greatly improved to

create a distributed learning platform, leveraging smartphones and personal devices. The last question is on the privacy implications and can be replied to by analyzing Section 2.3. In this section, we describe how the research is analyzing tools such as differential privacy to ensure the privacy of the data, in transit and on the devices. For the last question (Q4), we analyzed how Kubernetes can be used to create heterogeneous models that runs with different architectures such as the ubiquitous Intel 64-bit (called AMD64 or X86_64) and the nowadays extremely popular ARM architecture. We argue that this method could accelerate the development of applications based on federated learning, even on different architectures.

Author Contributions: Conceptualization, F.L.; Funding acquisition, B.M.; Investigation, F.L. and D.B.; Methodology, F.L.; Project administration, B.M.; Software, F.L.; Supervision, B.M.; Validation, D.B.; Visualization, D.B.; Writing—original draft, F.L.; Writing—review and editing, D.B. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study are openly available in Cifar repository at https://www.cs.toronto.edu/~kriz/cifar.html (accessed on 14 September 2024), in SVHN repository at http://ufldl.stanford.edu/housenumbers/ (accessed on 14 September 2024), in MNIST repository at https://yann.lecun.com/exdb/mnist/ (accessed on 14 September 2024) and in Fashion MNIST repository at http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/ (accessed on 14 September 2024).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Kairouz, P.; McMahan, H.B.; Avent, B.; Bellet, A.; Bennis, M.; Bhagoji, A.N.; Bonawitz, K.; Charles, Z.; Cormode, G.; Cummings, R.; et al. Advances and open problems in federated learning. *Found. Trends Mach. Learn.* **2021**, *14*, 1–210. [CrossRef]
2. Ye, M.; Fang, X.; Du, B.; Yuen, P.C.; Tao, D. Heterogeneous federated learning: State-of-the-art and research challenges. *ACM Comput. Surv.* **2023**, *56*, 1–44. [CrossRef]
3. Jere, S. Federated Learning in Mobile Edge Computing: An Edge-Learning Perspective for Beyond 5G. *arXiv* **2020**, arXiv:2007.08030. [CrossRef]
4. Parra-Ullauri, J.M.; Madhukumar, H.; Nicolaescu, A.C.; Zhang, X.; Bravalheri, A.; Hussain, R.; Vasilakos, X.; Nejabati, R.; Simeonidou, D. kubeFlower: A privacy-preserving framework for Kubernetes-based federated learning in cloud–edge environments. *Future Gener. Comput. Syst.* **2024**, *157*, 558–572. [CrossRef]
5. Kim, J.; Kim, D.; Lee, J. Design and implementation of Kubernetes enabled federated learning platform. In Proceedings of the 2021 IEEE International Conference on Information and Communication Technology Convergence (ICTC), Jeju Island, Republic of Korea, 20–22 October 2021; pp. 410–412.
6. Pham, K.Q.; Kim, T. Elastic Federated Learning with Kubernetes Vertical Pod Autoscaler for edge computing. *Future Gener. Comput. Syst.* **2024**, *158*, 501–515. [CrossRef]
7. Nikolaidis, F.; Symeonides, M.; Trihinas, D. Towards Efficient Resource Allocation for Federated Learning in Virtualized Managed Environments. *Future Internet* **2023**, *15*, 261. [CrossRef]
8. Jayaram, K.R.; Muthusamy, V.; Thomas, G.; Verma, A.; Purcell, M. Adaptive Aggregation For Federated Learning. In Proceedings of the 2022 IEEE International Conference on Big Data (Big Data), Osaka, Japan, 17–20 December 2022; pp. 180–185. [CrossRef]
9. Fang, H.; Qian, Q. Privacy Preserving Machine Learning with Homomorphic Encryption and Federated Learning. *Future Internet* **2021**, *13*, 94. [CrossRef]
10. Hu, R.; Guo, Y.; Li, H.; Pei, Q.; Gong, Y. Personalized Federated Learning With Differential Privacy. *IEEE Internet Things J.* **2020**, *7*, 9530–9539. [CrossRef]
11. Wei, K.; Li, J.; Ding, M.; Ma, C.; Yang, H.H.; Farokhi, F.; Jin, S.; Quek, T.Q.S.; Vincent Poor, H. Federated Learning with Differential Privacy: Algorithms and Performance Analysis. *IEEE Trans. Inf. Forensics Secur.* **2020**, *15*, 3454–3469. [CrossRef]
12. Ouadrhiri, A.E.; Abdelhadi, A. Differential Privacy for Deep and Federated Learning: A Survey. *IEEE Access* **2022**, *10*, 22359–22380. [CrossRef]
13. Jayaram, K.; Muthusamy, V.; Thomas, G.; Verma, A.; Purcell, M. Lambda FL: Serverless aggregation for federated learning. In Proceedings of the International Workshop on Trustable, Verifiable and Auditable Federated Learning, Vancouver, BC, Canada, 2 March 2022; Volume 9.

14. Benedict, S.; Saji, D.; Sukumaran, R.P.; Bhagyalakshmi, M. Blockchain-enabled federated learning on Kubernetes for air quality prediction applications. *J. Artif. Intell. Capsul. Netw.* **2021**, *3*, 196–217. [CrossRef]
15. Subramanya, T.; Riggio, R. Centralized and Federated Learning for Predictive VNF Autoscaling in Multi-Domain 5G Networks and Beyond. *IEEE Trans. Netw. Serv. Manag.* **2021**, *18*, 63–78. [CrossRef]
16. Chahoud, M.; Otoum, S.; Mourad, A. On the feasibility of Federated Learning towards on-demand client deployment at the edge. *Inf. Process. Manag.* **2023**, *60*, 103150. [CrossRef]
17. Hansmann, W.; Frank, M. On things to happen during a TCP handover. In Proceedings of the 28th Annual IEEE International Conference on Local Computer Networks, Bonn/Konigswinter, Germany, 20–24 October 2003; pp. 109–118.
18. Melin, P.; Monica, J.C.; Sanchez, D.; Castillo, O. Multiple ensemble neural network models with fuzzy response aggregation for predicting COVID-19 time series: The case of Mexico. *Healthcare* **2020**, *8*, 181. [CrossRef] [PubMed]
19. Muñoz-González, L.; Biggio, B.; Demontis, A.; Paudice, A.; Wongrassamee, V.; Lupu, E.C.; Roli, F. Towards poisoning of deep learning algorithms with back-gradient optimization. In Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security, Dallas, TX, USA, 3 November 2017; pp. 27–38.
20. Zhou, X.; Xu, M.; Wu, Y.; Zheng, N. Deep model poisoning attack on federated learning. *Future Internet* **2021**, *13*, 73. [CrossRef]
21. Chabanne, H.; Danger, J.L.; Guiga, L.; Kühne, U. Side channel attacks for architecture extraction of neural networks. *CAAI Trans. Intell. Technol.* **2021**, *6*, 3–16. [CrossRef]
22. Hu, H.; Salcic, Z.; Sun, L.; Dobbie, G.; Yu, P.S.; Zhang, X. Membership inference attacks on machine learning: A survey. *ACM Comput. Surv. (Csur.)* **2022**, *54*, 1–37. [CrossRef]
23. Ma, X.; Zhu, J.; Lin, Z.; Chen, S.; Qin, Y. A state-of-the-art survey on solving non-IID data in Federated Learning. *Future Gener. Comput. Syst.* **2022**, *135*, 244–258. [CrossRef]
24. Qu, Z.; Lin, K.; Li, Z.; Zhou, J. Federated learning's blessing: Fedavg has linear speedup. In Proceedings of the ICLR 2021-Workshop on Distributed and Private Machine Learning (DPML), Vienna, Austria, 4 May 2021.
25. Huang, Z.; Hu, R.; Guo, Y.; Chan-Tin, E.; Gong, Y. DP-ADMM: ADMM-based distributed learning with differential privacy. *IEEE Trans. Inf. Forensics Secur.* **2019**, *15*, 1002–1012. [CrossRef]
26. Lindell, Y. Secure multiparty computation. *Commun. ACM* **2020**, *64*, 86–96. [CrossRef]
27. Duan, Q.; Huang, J.; Hu, S.; Deng, R.; Lu, Z.; Yu, S. Combining Federated Learning and Edge Computing toward Ubiquitous Intelligence in 6G Network: Challenges, Recent Advances, and Future Directions. *IEEE Commun. Surv. Tutor.* **2023**, *25*, 2892–2950. [CrossRef]
28. Beutel, D.J.; Topal, T.; Mathur, A.; Qiu, X.; Fernandez-Marques, J.; Gao, Y.; Sani, L.; Li, K.H.; Parcollet, T.; de Gusmão, P.P.B.; et al. Flower: A friendly federated learning framework. Available online: https://hal.science/hal-03601230/document (accessed on 14 September 2024).
29. Shamsian, A.; Navon, A.; Fetaya, E.; Chechik, G. Personalized federated learning using hypernetworks. In Proceedings of the International Conference on Machine Learning. PMLR, Virtual Event, 18–24 July 2021; pp. 9489–9502.
30. Wang, J.; Li, Y.; Ye, R.; Li, J. High Precision Method of Federated Learning Based on Cosine Similarity and Differential Privacy. In Proceedings of the 2022 IEEE International Conferences on Internet of Things (iThings) and IEEE Green Computing & Communications (GreenCom) and IEEE Cyber, Physical & Social Computing (CPSCom) and IEEE Smart Data (SmartData) and IEEE Congress on Cybermatics (Cybermatics), Espoo, Finland, 22–25 August 2022; pp. 533–540. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.